

7.6 Sorting a Positional List

In Section 3.1.2, we introduced the *insertion-sort* algorithm in the context of an array-based sequence. In this section, we develop an implementation that operates on a `PositionalList`, relying on the same high-level algorithm in which each element is placed relative to a growing collection of previously sorted elements.

We maintain a variable named `marker` that represents the rightmost position of the currently sorted portion of a list. During each pass, we consider the position just past the `marker` as the `pivot` and consider where the pivot's element belongs relative to the sorted portion; we use another variable, named `walk`, to move leftward from the `marker`, as long as there remains a preceding element with value larger than the pivot's. A typical configuration of these variables is diagrammed in Figure 7.9. A Java implementation of this strategy is given in Code 7.15.

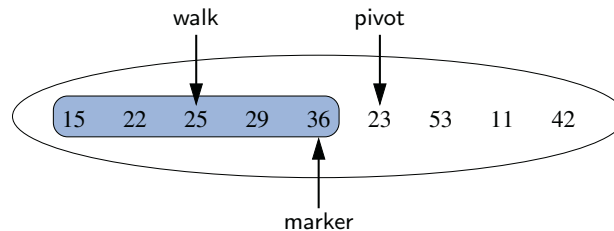


Figure 7.9: Overview of one step of our insertion-sort algorithm. The shaded elements, those up to and including `marker`, have already been sorted. In this step, the pivot's element should be relocated immediately before the `walk` position.

```

1  /** Insertion-sort of a positional list of integers into nondecreasing order */
2  public static void insertionSort(PositionalList<Integer> list) {
3      Position<Integer> marker = list.first();    // last position known to be sorted
4      while (marker != list.last()) {
5          Position<Integer> pivot = list.after(marker);
6          int value = pivot.getElement();          // number to be placed
7          if (value > marker.getElement())         // pivot is already sorted
8              marker = pivot;
9          else {                                   // must relocate pivot
10             Position<Integer> walk = marker;     // find leftmost item greater than value
11             while (walk != list.first() && list.before(walk).getElement() > value)
12                 walk = list.before(walk);
13             list.remove(pivot);                   // remove pivot entry and
14             list.addBefore(walk, value);          // reinsert value in front of walk
15         }
16     }
17 }

```

Code Fragment 7.15: Java code for performing insertion-sort on a positional list.